

Research Notes on K-nearest neighbors algorithm

K-nearest neighbors algorithm

>> Section 1

for constants B_1 and B_2 where $s_n^2 = \sum_{i=1}^n w_{ni}^2$ and $t_n = n - 2/d \sum_{i=1}^n w_{ni} \{i + 2/d - (i-1)1 + 2/d\}$. The optimal weighting scheme $\{w_{ni}\}_{i=1}^n$, that balances the two terms in the display above, is given as follows: set $k^* = \lfloor B n^{\frac{4}{d+4}} \rfloor$,

>> Section 2

A commonly used distance metric for continuous variables is Euclidean distance. For discrete variables, such as for text classification, another metric can be used, such as the overlap metric (or Hamming distance). In the context of gene expression microarray data, for example, k-NN has been employed with correlation coefficients, such as Pearson and Spearman, as a metric. Often, the classification accuracy of k-NN can be improved significantly if the distance metric is learned with specialized algorithms such as large margin nearest neighbor or neighborhood components analysis.

>> Section 3

Parameter selection The best choice of k depends upon the data; generally, larger values of k reduces effect of the noise on the classification, but make boundaries between classes less distinct. A good k can be selected by various heuristic techniques (see hyperparameter optimization). The special case where the class is predicted to be the class of the closest training sample (i.e. when k = 1) is called the nearest neighbor algorithm. The accuracy of the k-NN algorithm can be severely degraded by the presence of noisy or irrelevant features, or if the feature scales are not consistent with their importance. Much research effort has been put into selecting or scaling features to improve classification. A particularly popular approach is the use of evolutionary algorithms to optimize feature scaling. Another popular approach is to scale features by the mutual information of the training data with the training classes. In binary (two class) classification problems, it is helpful to choose k to be an odd number as this avoids tied votes. One popular way of choosing the empirically optimal k in this setting is via bootstrap method.

>> Section 4

In k-NN regression, also known as k-NN smoothing, the k-NN algorithm is used for estimating continuous variables. One such algorithm uses a weighted average of the k nearest neighbors, weighted by the inverse of their

distance. This algorithm works as follows:

>> Section 5

$R^* \leq R_{kNN} \leq R^* (2 - MR^* M - 1)$ taking values in $R^d \times \{1, 2\}$, where Y is the class label of X, so that $X|Y=r \sim P_r$ for $r = 1, 2$ (and probability distributions P_r). Given some norm $\cdot$ on R^d and a point $x \in R^d$, let $(X_1, Y_1), \dots, (X_n, Y_n)$ be a reordering of the training data such that $\|X_1 - x\| \leq \|X_n - x\|$.

>> Section 6

Statistical setting Suppose we have pairs $(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)$ taking values in $R^d \times \{1, 2\}$, where Y is the class label of X, so that $X|Y=r \sim P_r$ for $r = 1, 2$ (and probability distributions P_r). Given some norm $\cdot$ on R^d and a point $x \in R^d$, let $(X_1, Y_1), \dots, (X_n, Y_n)$ be a reordering of the training data such that $\|X_1 - x\| \leq \|X_n - x\|$.

>> Section 7

The training examples are vectors in a multidimensional feature space, each with a class label. The training phase of the algorithm consists only of storing the feature vectors and class labels of the training samples. In the classification phase, k is a user-defined constant, and an unlabeled vector (a query or test point) is classified by assigning the label which is most frequent among the k training samples nearest to that query point.

>> Section 8

Feature extraction When the input data to an algorithm is too large to be processed and it is suspected to be redundant (e.g. the same measurement in both feet and meters) then the input data will be transformed into a reduced representation set of features (also named features vector). Transforming the input data into the set of features is called feature extraction. If the features extracted are carefully chosen it is expected that the features set will extract the relevant information from the input data in order to perform the desired task using this reduced representation instead of the full size input. Feature extraction is performed on raw data prior to applying k-NN algorithm on the transformed data in feature space. An example of a typical computer vision computation pipeline for face recognition using k-NN including feature extraction and dimension reduction pre-processing steps (usually implemented with OpenCV):

Research Notes on K-nearest neighbors algorithm

K-nearest neighbors algorithm

>> Section 9

Metric learning The K-nearest neighbor classification performance can often be significantly improved through (supervised) metric learning. Popular algorithms are neighbourhood components analysis and large margin nearest neighbor. Supervised metric learning algorithms use the label information to learn a new metric or pseudo-metric.

>> Section 10

A drawback of the basic "majority voting" classification occurs when the class distribution is skewed. That is, examples of a more frequent class tend to dominate the prediction of the new example, because they tend to be common among the k nearest neighbors due to their large number. One way to overcome this problem is to weight the classification, taking into account the distance from the test point to each of its k nearest neighbors. The class (or value, in regression problems) of each of the k nearest points is multiplied by a weight proportional to the inverse of the distance from that point to the test point. Another way to overcome skew is by abstraction in data representation. For example, in a self-organizing map (SOM), each node is a representative (a center) of a cluster of similar points, regardless of their density in the original training data. k -NN can then be applied to the SOM.